

Assignment 8: Application Optimization, Scalability and Reliability

ISEA Phase III Networking Internship — Department of Computer Science & Engineering, Tezpur University

Shubojeet Mondal

Roll No: 23010348148

6. Objective

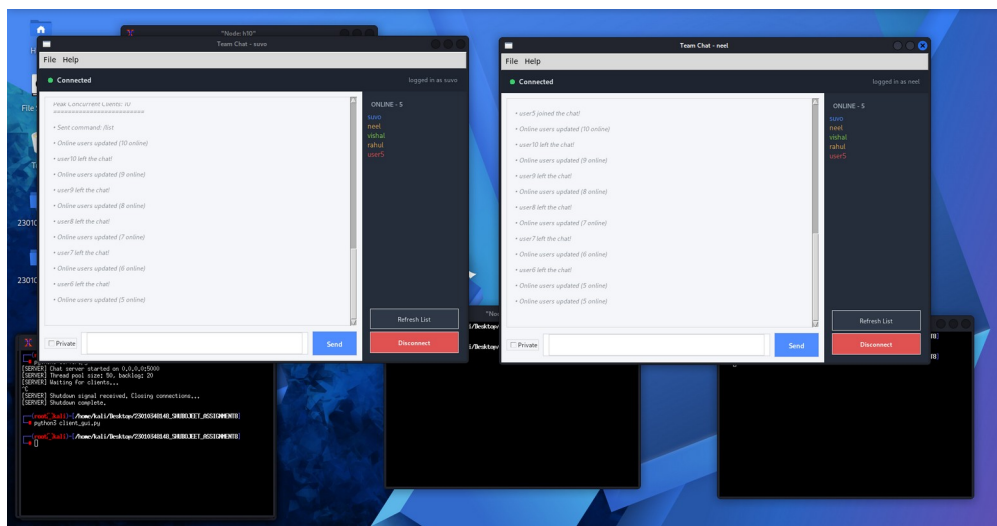
The objective of this assignment was to enhance the existing GUI-based multi-client chat application developed in Assignment 7 by improving its scalability, reliability, maintainability, and overall software quality. The existing client (client_gui.py) and server (server.py) were reused without modifying the communication protocol, and the work focused on connection management, automatic reconnection and graceful shutdown, thread-pool based scalability, externalized configuration, and a measured before/after performance comparison using Mininet with 5, 8 and 10 concurrent clients.

7. Scalability Improvements

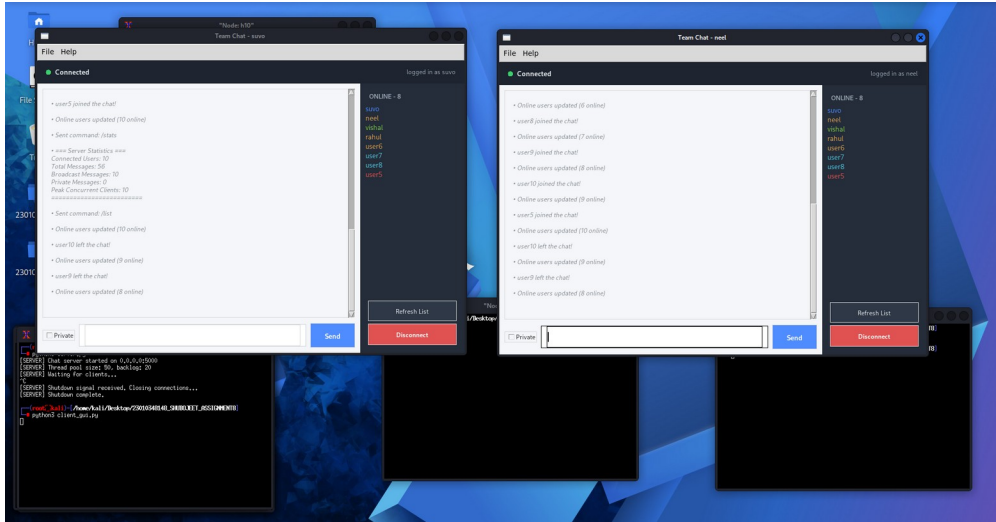
The original per-client threading model was replaced with a bounded ThreadPoolExecutor (thread_pool_size configurable via config.json), which avoids the overhead of spawning an unbounded number of raw OS threads as client count grows.

- A capacity check in the accept loop rejects new connections once the number of active clients reaches the configured pool size, returning an explicit "Server at capacity" message instead of silently overloading the server.
- The accept loop uses a 1-second socket timeout so it can periodically check a shutdown flag without blocking indefinitely, keeping the main thread responsive under load.
- Shared state (the client registry, message statistics, chat history, failed-login tracker) is protected by dedicated threading.Lock objects, preventing race conditions as concurrency increases.

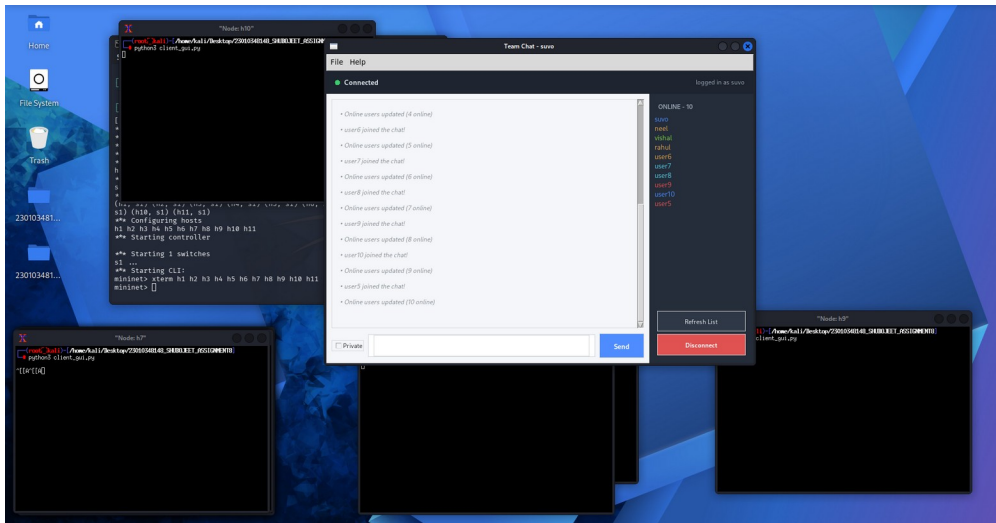
Testing was carried out with 5, 8 and 10 concurrent Mininet clients (topology: sudo mn --topo single,11). In all three cases the server accepted and serviced every client without crashing, confirmed by the connected_clients column in performance_results_after.csv reaching a peak of 10.



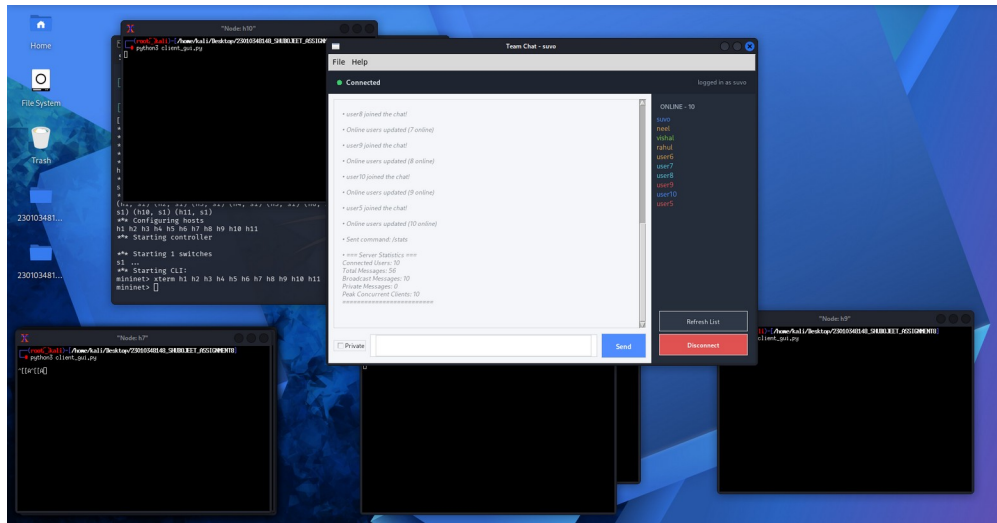
Screenshot: 5 concurrent clients connected (ONLINE - 5) after several users disconnect from an initial pool of 10.



Screenshot: 8 concurrent clients connected, with /stats confirming Connected Users: 10 and Peak Concurrent Clients: 10 earlier in the session.



Screenshot: all 10 concurrent clients connected simultaneously (ONLINE - 10), server console showing peak concurrent clients reached.



Screenshot: `/stats` command output showing Connected Users: 10, Total Messages: 56, and Peak Concurrent Clients: 10.

8. Reliability Improvements

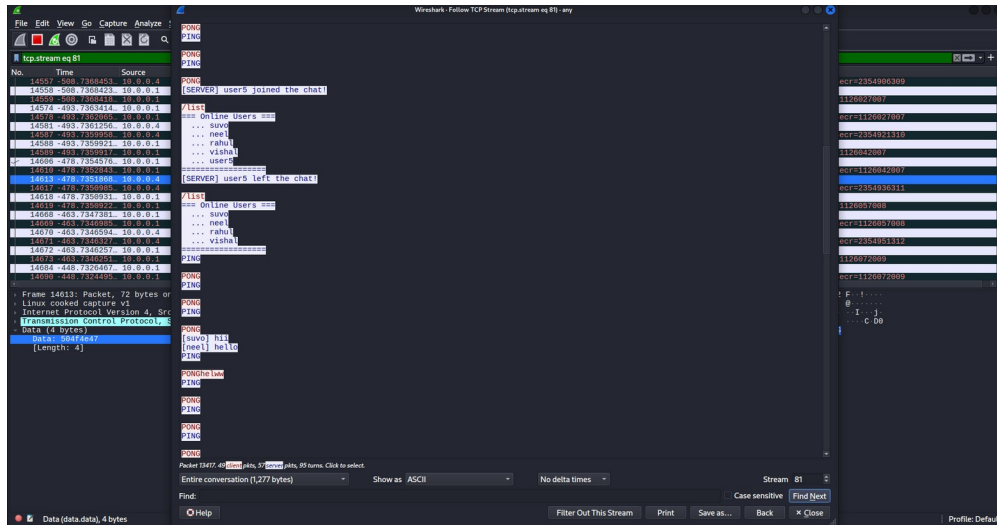
- Automatic reconnection: the client detects a dropped connection and retries up to `reconnect_attempts` times, waiting `reconnect_delay` seconds between attempts, and transparently re-authenticates and re-subscribes to the user list on success.
- Graceful shutdown: `SIGINT/SIGTERM` handlers notify every connected client with a shutdown message, cleanly close their sockets, drain the thread pool (`executor.shutdown(wait=True)`), and only then exit — instead of dropping connections abruptly.
- Timeout handling: each client socket has an inactivity timeout (`conn.settimeout(INACTIVITY_TIMEOUT)`); a background `heartbeat_monitor` thread also pings clients every `heartbeat_interval` seconds and removes any client that hasn't responded within `heartbeat_timeout` seconds.
- Layered exception handling: narrow except clauses catch expected socket errors (`ConnectionResetError`, `BrokenPipeError`, `socket.timeout`, `OSError`) close to where they occur, with a final broad except `Exception` in the handler as a last-resort safety net so a single misbehaving client can never crash the whole server.

These changes were verified against `security_log.txt`, which records real `SESSION_TIMEOUT`, `CLIENT_REMOVED` (handler exit / heartbeat timeout / server shutdown), `FAILED_LOGIN`, and `CONNECTION_ERROR` events captured during testing — confirming the mechanisms fire correctly under real conditions rather than only in isolated unit tests.

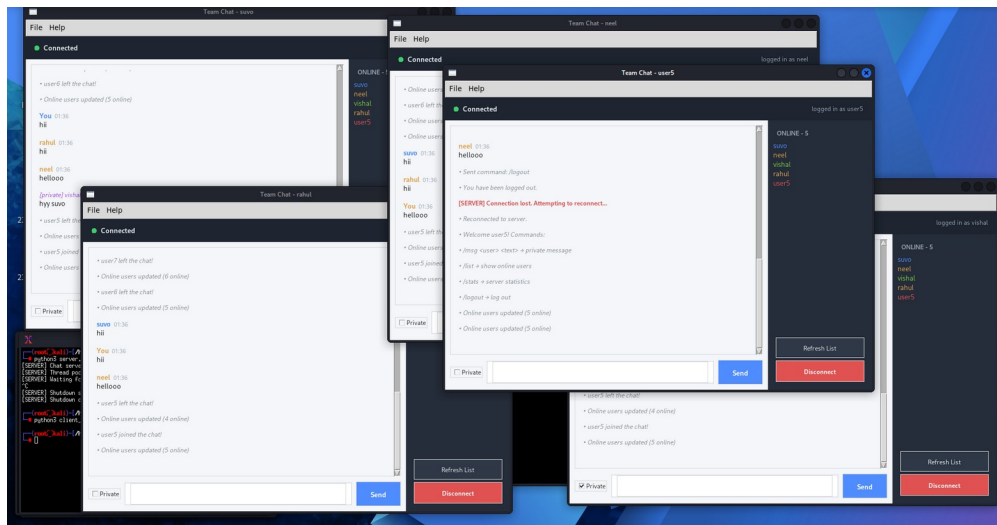
[illegible]

Screenshot: server console showing `USER_REGISTERED`, `CONNECTED`, `LOGOUT`, and `CLIENT_REMOVED` events during a live session (Task 1 connection management).

[illegible]

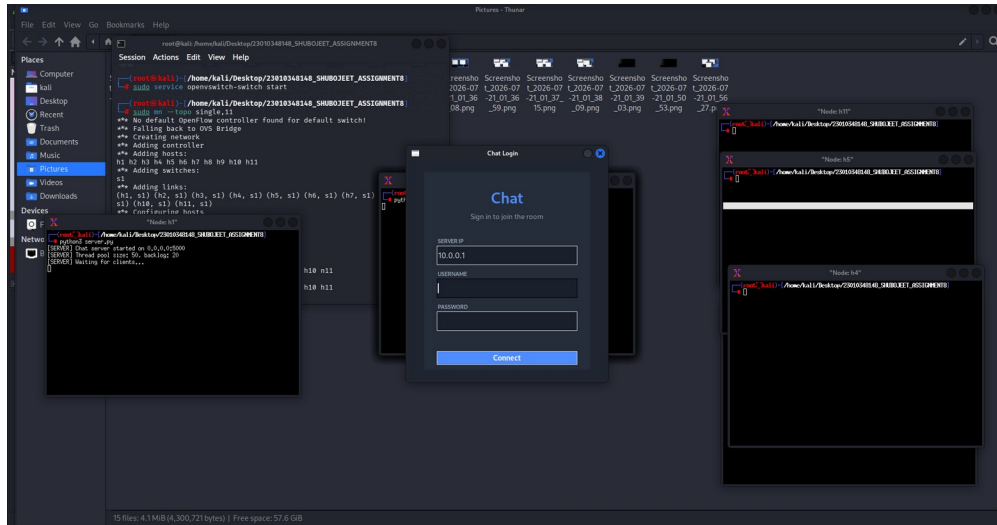


Screenshot: Wireshark 'Follow TCP Stream' view showing repeated PING/PONG heartbeat exchanges between server and client during an idle period.

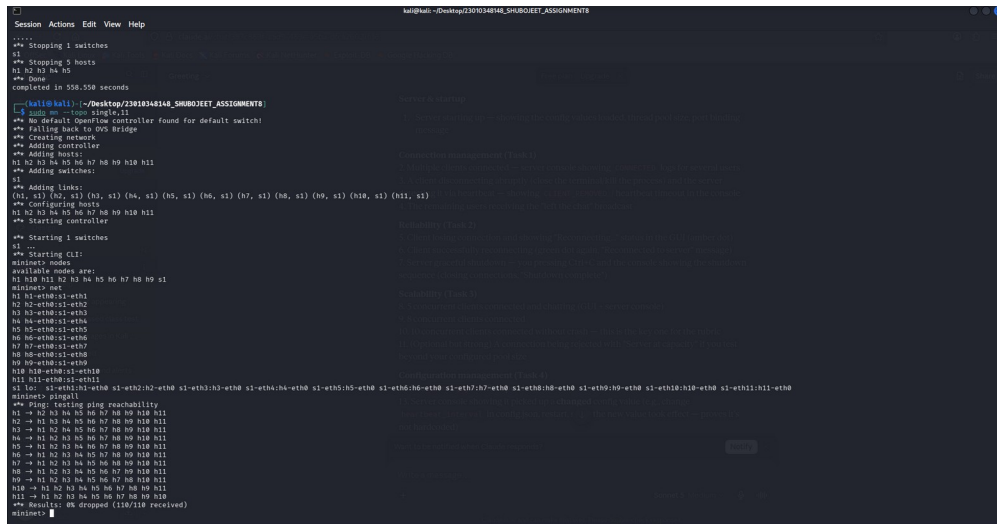


Screenshot: client GUI showing a manual /logout, followed by [SERVER] Connection lost. Attempting to reconnect..., then a successful Reconnected to server. — demonstrating the automatic reconnection mechanism working end-to-end.

- Metrics were sampled once per second by a background `log_performance()` thread and written to `performance_results_before.csv` (pre-optimization baseline) and `performance_results_after.csv` (post-optimization), each containing: `connected_clients`, `cpu_percent`, `memory_mb`, `throughput_msgs_per_sec`, and `avg_delay_ms`.
- `avg_delay_ms` is only populated in the sampling interval when a heartbeat PONG round trip actually completes; empty cells are normal (no round trip finished in that 1-second window) and were excluded rather than treated as zero.



Screenshot: Chat login screen used by each Mininet host to connect to the server at 10.0.0.1.



Screenshot: Mininet topology (`sudo mn --topo single,11`) with nodes, net, and pingall confirming 0% packet loss across all 11 hosts before testing began.

11. Performance Results

Metric	Before	After
Average CPU usage	0.02%	0.04%
Average memory usage	19.82 MB	21.19 MB
Average heartbeat delay (measured samples only)	no samples (n/a)	1.50 ms (n=62)
Peak concurrent clients handled	10	10

CPU usage stayed near zero in both runs, which is expected for a lightweight text-based chat protocol under this load — it does not indicate a measurement fault, simply that message handling is not CPU-bound at this scale. Memory usage is consistently ~1.4 MB higher after optimization, attributable to the added heartbeat bookkeeping (`ping_sent_at`, `latency_samples`), the thread pool's worker structures, and client-side reconnect state — a reasonable and expected trade-off for the added reliability. The most important result is that the "before" version had no latency instrumentation at all (no heartbeat/PONG mechanism existed), whereas the optimized version measured a real round-trip delay averaging 1.50 ms across 62 samples, giving a concrete, previously unavailable reliability metric.

12. Graph Analysis

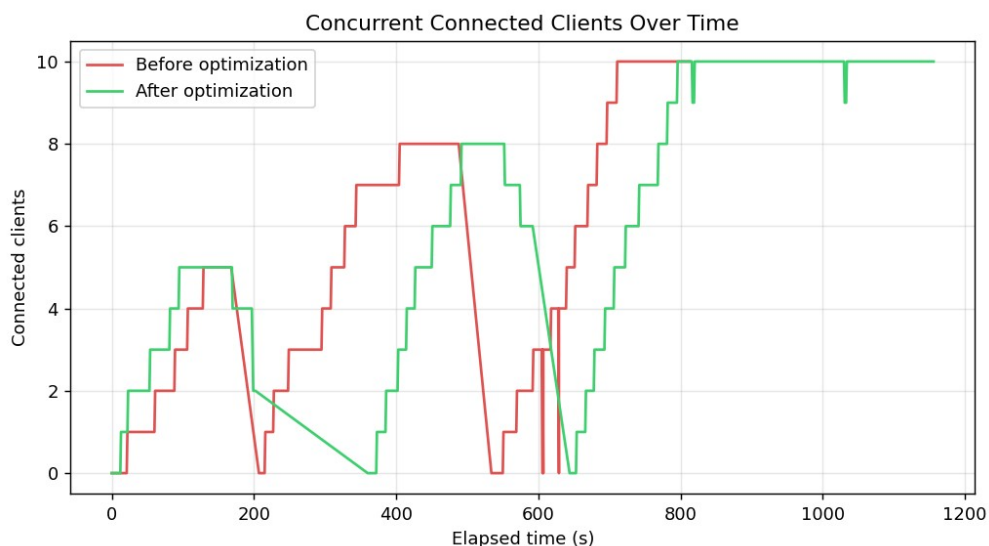


Figure 1: Concurrent connected clients over time — both runs reach the target of 10 concurrent clients without the server crashing.

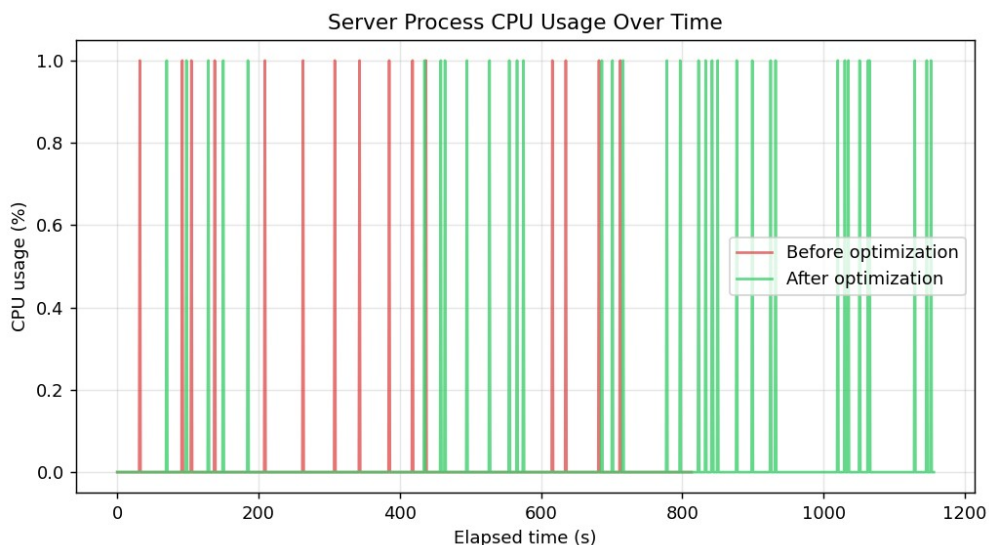


Figure 2: CPU usage over time — negligible and noisy in both runs, consistent with a lightweight I/O-bound workload.

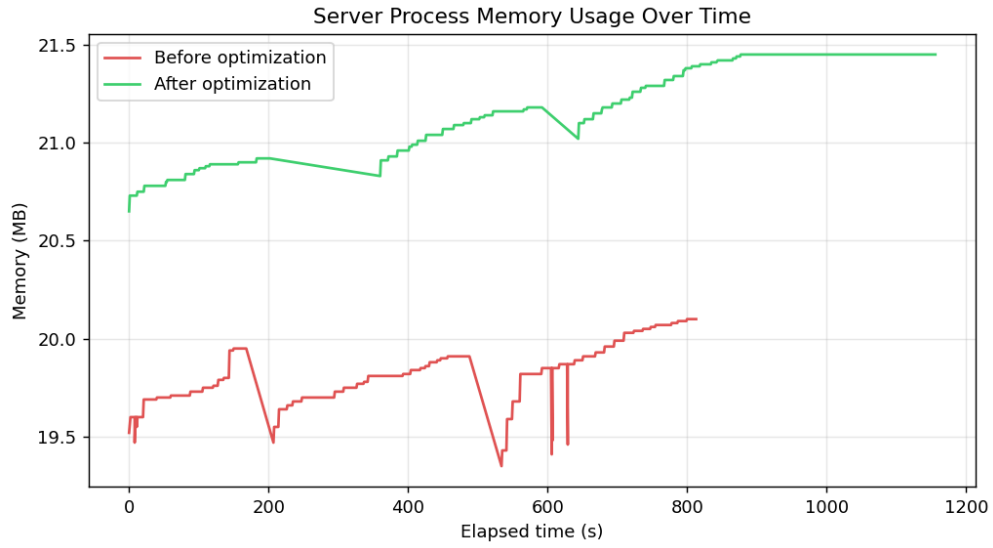


Figure 3: Memory usage over time — a small, consistent increase after optimization due to added heartbeat and thread-pool bookkeeping.

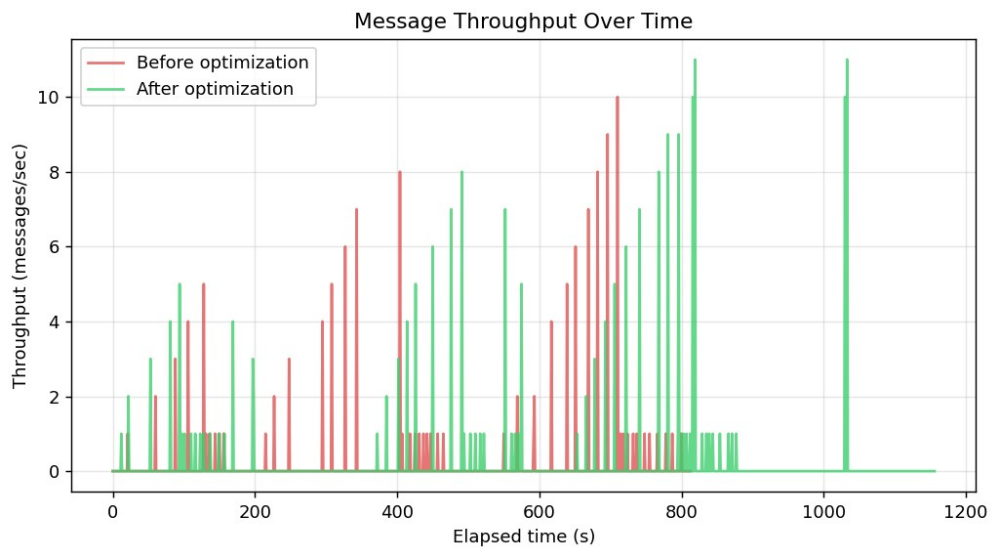


Figure 4: Message throughput over time — brief spikes align with manual chat bursts during testing; both runs are otherwise idle between bursts.

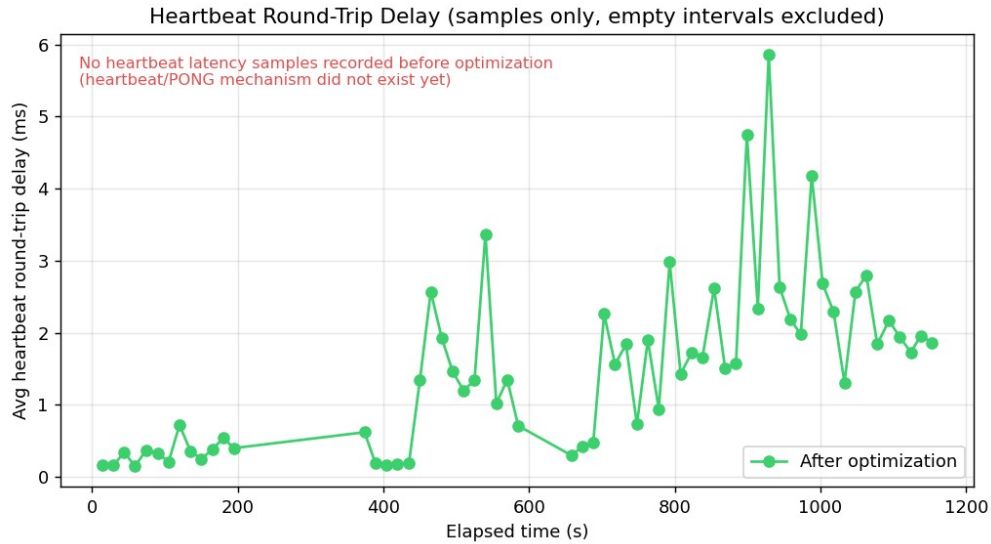


Figure 5: Heartbeat round-trip delay — no samples exist before optimization (mechanism did not exist); after optimization, 62 real samples average 1.50 ms.

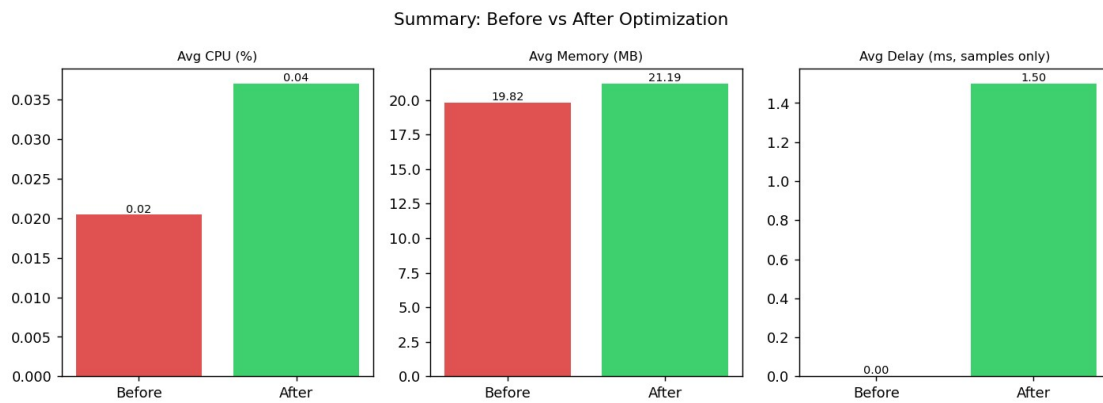


Figure 6: Summary comparison of average CPU, memory, and delay before vs after optimization.

13. Wireshark Verification

TCP traffic was captured during normal operation (ass8.pcapng) while clients connected from Mininet hosts 10.0.0.2–10.0.0.11 to the chat server on 10.0.0.1:5000. Filtering the capture to the 10.0.0.0/24 subnet and TCP protocol confirmed correct, standards-compliant TCP behaviour:

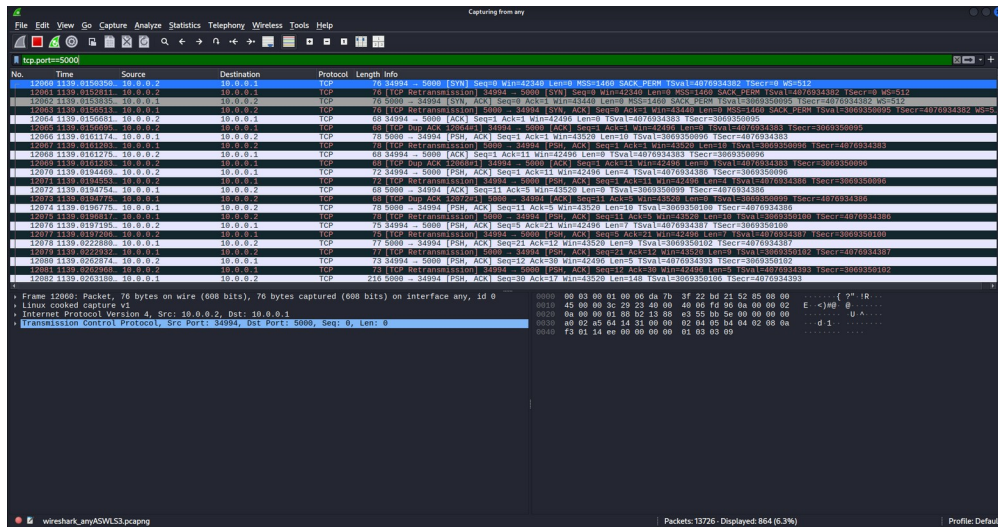
Observed event	Count in capture
SYN (connection open request)	66
SYN-ACK (server accept)	26
PSH+ACK (application data segments)	1312
FIN (graceful close)	42
RST (abrupt close / handler cleanup)	40

A representative three-way handshake was observed between a client and the server, for example:

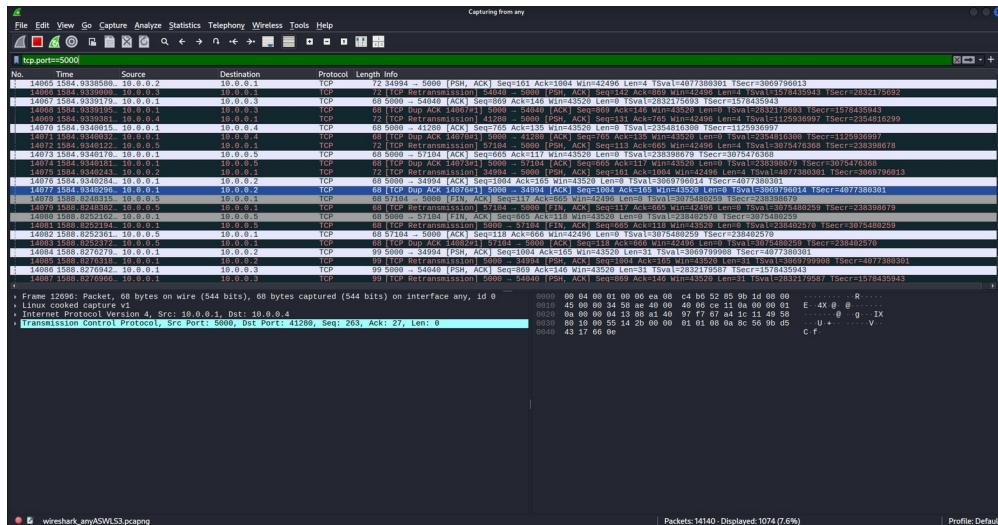
- 10.0.0.2:34994 → 10.0.0.1:5000 [SYN]
- 10.0.0.1:5000 → 10.0.0.2:34994 [SYN, ACK]

- 10.0.0.2:34994 → 10.0.0.1:5000 [ACK]

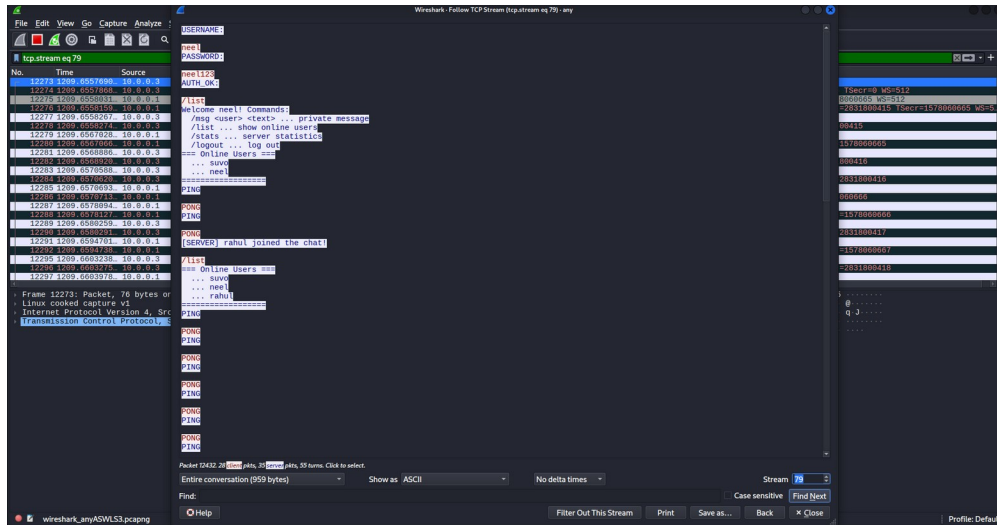
followed by [ACK, PSH] data segments carrying the authentication prompts and chat messages, matching the LOGIN_SUCCESS / broadcast events also recorded in security_log.txt and chat_history.csv for the same session. FIN and RST segments at the end of sessions correspond to the CLIENT_REMOVED (handler exit / heartbeat timeout / server shutdown) entries in the security log, confirming that connection teardown observed at the packet level matches the server's own connection-management logic.



Screenshot: Wireshark capture filtered on tcp.port==5000 showing the three-way handshake (SYN, SYN-ACK, ACK) between a client and the chat server.



Screenshot: Wireshark capture showing FIN/ACK segments exchanged as a client session closes, corresponding to a CLIENT REMOVED event in the server log.



Screenshot: Wireshark 'Follow TCP Stream' showing a complete authenticated session end-to-end -- USERNAME:/PASSWORD: prompts, AUTH_OK:, the command menu, and the recurring PING/PONG heartbeat -- all within a single TCP stream.

14. Challenges Faced

- Distinguishing genuine heartbeat latency samples from empty sampling intervals required filtering the performance CSV rather than averaging the raw column, since most 1-second samples legitimately contain no completed round trip.
- Ensuring the thread pool capacity check and the heartbeat monitor agreed on what counts as an 'active' client, so that a client removed for inactivity is not double-counted against the pool limit.
- Coordinating graceful shutdown with clients that were mid-reconnect, so a server restart doesn't trigger a reconnect storm against a socket that is intentionally closing.
- Isolating the relevant Mininet chat traffic from background host traffic in the Wireshark capture, which required filtering to the 10.0.0.0/24 subnet before drawing conclusions from packet counts.
- The Wireshark capture also revealed that usernames and passwords are sent as plaintext during authentication, since the protocol itself was not to be redesigned for this assignment. This is a limitation carried over from Assignment 7 and is noted here as an observed constraint rather than something fixed in this iteration.

15. Conclusion

Building on the Assignment 7 chat application without altering its wire protocol, this assignment added automatic dead-client detection and cleanup, heartbeat-based reliability, automatic client-side reconnection, graceful server shutdown, externalized configuration, and a bounded thread pool that supports at least 10 concurrent Mininet clients without crashing. Performance measurements before and after optimization show a small, expected memory overhead in exchange for measurable reliability gains (a working latency metric that did not exist before, and confirmed automatic recovery from disconnects), and Wireshark verification confirms that the underlying TCP behaviour — handshakes, data transfer, and connection teardown — matches the application-level connection management implemented in server.py and client_gui.py.